SIMATS Engineering

SIMATS Online Programming Lab

CO5 - AT2: Application Based Problem Solving

Employee Management and Payroll Application using Java AWT and MySQL

Student Name	I. MD IMTHIAZ
Register Number	192521360
Subject Code	CSA0925
Subject Name	Java Programming
Assignment	CO5-AT2 - Application Based Problem Solvin (Q1)
Maximum Marks	25

1. Requirement Analysis and Database Design

1.1 Problem Statement

A medium-sized organization currently manages employee records and payroll manually using spreadsheets, which leads to duplicate records, inconsistent data, and difficulty retrieving employee information. A desktop-based Employee Management and Payroll Application is proposed, built using Java AWT for the graphical user interface and MySQL for persistent storage, so that authorized HR staff can add, view, search, update, and delete employee records, and automatically calculate payroll.

1.2 Functional Requirements

- Add (register) a new employee with personal, contact, department, and salary details.
- Search employees by Employee ID, department, designation, or employment status.
- Update an existing employee's details.
- Delete an employee record after confirmation.
- View a complete list of all registered employees.
- Calculate Gross Salary and Net Salary for one employee or for all employees.
- Validate mandatory fields, numeric salary values, and duplicate Employee IDs before any database write.
- Provide user feedback (dialog boxes) after every database operation, including error conditions.

1.3 Non-Functional Requirements

- Usability: simple AWT-based forms with clear labels and validation messages.
- Reliability: all database operations wrapped in try-catch with meaningful error dialogs instead of crashes.
- Data consistency: foreign key from employee to department, unique constraints on Employee ID and email.
- Maintainability: layered design (Model / DAO / GUI) so new modules (attendance, leave, appraisal) can be added without redesigning the existing application.

1.4 Entity-Relationship Design

Two related tables are used: **department** (parent) and **employee** (child), connected by a one-to-many relationship (one department has many employees) through the foreign key `employee.department_id` → `department.department_id`.

Table: department

Column	Type	Constraint
department_id	INT (AUTO_INCREMENT)	PRIMARY KEY
department_name	VARCHAR(50)	NOT NULL, UNIQUE

Table: employee

Column	Type	Constraint
employee_id	VARCHAR(15)	PRIMARY KEY

Column	Type	Constraint
name	VARCHAR(100)	NOT NULL
gender	VARCHAR(10)	-
date_of_birth	DATE	-
contact_number	VARCHAR(15)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE
address	VARCHAR(255)	-
department_id	INT	NOT NULL, FOREIGN KEY -> department
designation	VARCHAR(50)	-
date_of_joining	DATE	NOT NULL
basic_salary	DECIMAL(12,2)	NOT NULL, CHECK >= 0
allowances	DECIMAL(12,2)	DEFAULT 0
deductions	DECIMAL(12,2)	DEFAULT 0
employment_status	VARCHAR(15)	NOT NULL, CHECK IN (ACTIVE/INACTIVE/ON_LEA\

2. GUI Design using Java AWT

The application is a single AWT `Frame` using `CardLayout` to switch between six screens without opening multiple windows. Each screen is built from AWT components as required by the specification:

Screen	Purpose	Key AWT components used
Employee Registration	Add a new employee	Label, TextField, TextArea, Choice, Butt n
Employee Search	Search by department / designation / stat	sC/hloDice, TextField, Button, TextArea (res ults)
Employee Update	Load and edit an existing employee	TextField, Choice, TextArea, Button
Employee Delete	Preview and delete a record	TextField, Button, Dialog (confirmation)
Employee List / View	Tabular listing of all employees	TextArea, Button (Refresh)
Payroll / Salary Calculation	Gross / Net salary for one or all employee	TextField, Button, TextArea

Event handling is implemented using `ActionListener` (button clicks) attached either via lambda expressions or the Frame's own `actionPerformed()` method. Modal `Dialog` boxes are used for delete confirmation and for all success/error feedback messages, satisfying the requirement that the application provide feedback after every database operation.

3. JDBC Connectivity and CRUD Implementation

All database access is centralised in two classes: `DBConnectionUtil` (loads the MySQL driver and opens a `Connection` to `employee_payroll_db`) and `EmployeeDAO` (all CRUD logic). Every SQL statement uses `PreparedStatement` with bound parameters, which prevents SQL injection and correctly handles numeric/date types.

Operation	DAO Method	SQL Used
Create	insertEmployee(Employee)	INSERT INTO employee (...) VALUES (...)
Retrieve (by ID)	getEmployeeById(String)	SELECT ... JOIN department WHERE empl yee_id = ?
Retrieve (search)	searchEmployees(dept, designation, status)	SELECT ... WHERE dynamic filters
Retrieve (all)	getAllEmployees()	SELECT ... (no filters)
Update	updateEmployee(Employee)	UPDATE employee SET ... WHERE employ ee_id = ?
Delete	deleteEmployee(String)	DELETE FROM employee WHERE employ e_id = ?

4. Payroll and Validation Module

The Payroll screen calculates, for one employee or for all employees:

Gross Salary = Basic Salary + Allowances

Net Salary = Gross Salary - Deductions

These formulas are implemented as `getGrossSalary()` and `getNetSalary()` methods on the `Employee` model using `BigDecimal` arithmetic to avoid floating-point rounding errors in monetary values.

4.1 Validation Rules (enforced in EmployeeDAO.validate())

- Employee ID, Name, Contact Number, Email, Department and Date of Joining are mandatory.
- Contact number must match a 10-digit numeric pattern.
- Email must match a standard email regular expression.
- Date of joining cannot be a future date.
- Basic salary must be numeric and cannot be negative; Allowances/Deductions cannot be negative.
- Employee ID must be unique - checked against the database before insert (duplicate Employee ID error).
- Employment status must be one of ACTIVE, INACTIVE, ON_LEAVE, TERMINATED.

5. Testing, Exception Handling and Evaluation

The application distinguishes two custom checked exceptions - `EmployeeValidationException` (bad input) and `EmployeeNotFoundException` (operation on a non-existent ID) - from `SQLException` (database/connection failures). Each is caught separately in the GUI layer and reported through a Dialog, so the application never terminates abnormally.

#	Test Case	Input / Action	Expected Result
1	New employee registration	Valid, unique Employee ID and all mar	datatory fields	Record inserted; success dialog shown
2	Duplicate employee registrat	oEnmployee ID that already exists	EmployeeValidationException - 'Dupli	ate Employee ID' er
3	Employee search	Existing department / designation keyw	oMatching rows listed in results TextAre	a
4	Employee update	Load existing ID, change salary/depar	mRent, suapvdeated in database; confirmation	dialog
5	Employee deletion	Existing Employee ID, confirm Yes in (	aRlogw removed after confirmation dialog	
6	Employee not found	Update/Delete/Search with a non-exis	nEtmIDployeeNotFoundException - 'No er	ployee found' error
7	Invalid input	Non-numeric salary or malformed date	EmployeeValidationException - specif	c field error dialog
8	Empty mandatory fields	Submit form with Name or Email left b	nVkalidation error before any SQL is exe	cuted
9	Salary calculation	Any valid employee ID on Payroll scre	nGross = Basic + Allowances; Net = Gr	ss - Deductions dis
10	Database connection failure	MySQL service stopped / wrong URL	SQLException caught; 'Database erro	' dialog, app remain

The application was evaluated against usability (clear labelled forms and dialogs), correctness (payroll formulas verified against manual calculation), reliability (all DB calls wrapped in try-catch), database consistency (foreign key + unique constraints enforced at both application and schema level), and maintainability (DAO layer isolates SQL from the GUI, so future modules such as attendance or leave management can reuse the same Employee model and connection utility).

6. Expected Deliverable

A complete, working Employee Management and Payroll desktop application built with Java AWT (GUI), Java Event Handling (user interaction), JDBC (MySQL connectivity), MySQL Workbench/MySQL (database), OOP concepts (Employee model, EmployeeDAO, encapsulation via getters/setters), custom exception handling, and SQL for all persistence operations. Source code, the MySQL schema script, and this report (requirement analysis, ER design, GUI design, CRUD implementation, payroll formulas, validation rules and test cases) are included as the submission.

7. Source Code Listing

The full source code below implements the design described in Sections 1-4 (schema.sql + 6 Java classes).

schema.sql

```
-- ============================================================
-- Employee Management and Payroll Application
-- Database: MySQL
-- Author  : G Sai Deepak (Reg No: 192311432)
-- Subject : CSA0925 - Java Programming (CO5-AT2)
-- ============================================================

CREATE DATABASE IF NOT EXISTS employee_payroll_db;
USE employee_payroll_db;

-- ------------------------------------------------------------------------------
-- Table: department  (parent table)
-- ------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS department (
    department_id    INT AUTO_INCREMENT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL UNIQUE
);

INSERT INTO department (department_name) VALUES
    ('HR'), ('Finance'), ('IT'), ('Sales'), ('Operations')
ON DUPLICATE KEY UPDATE department_name = department_name;

-- ------------------------------------------------------------------------------
-- Table: employee    (child table, FK -> department)
-- ------------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS employee (
    employee_id       VARCHAR(15)    NOT NULL PRIMARY KEY,
    name              VARCHAR(100)  NOT NULL,
    gender            VARCHAR(10),
    date_of_birth     DATE,
    contact_number    VARCHAR(15)    NOT NULL,
    email             VARCHAR(100)  NOT NULL UNIQUE,
    address           VARCHAR(255),
    department_id     INT            NOT NULL,
    designation       VARCHAR(50),
    date_of_joining   DATE           NOT NULL,
    basic_salary      DECIMAL(12,2) NOT NULL,
    allowances        DECIMAL(12,2) DEFAULT 0,
    deductions        DECIMAL(12,2) DEFAULT 0,
    employment_status VARCHAR(15)    NOT NULL DEFAULT 'ACTIVE',
    CONSTRAINT fk_employee_department
        FOREIGN KEY (department_id) REFERENCES department(department_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_salary_positive CHECK (basic_salary >= 0),
    CONSTRAINT chk_status CHECK (employment_status IN ('ACTIVE','INACTIVE','ON_LEAVE','TERMINATED'))
);

CREATE INDEX idx_employee_department ON employee(department_id);
CREATE INDEX idx_employee_status ON employee(employment_status);

-- ------------------------------------------------------------------------------
-- Sample data (for demonstration / testing)
-- ------------------------------------------------------------------------------
INSERT INTO employee
(employee_id, name, gender, date_of_birth, contact_number, email, address,
 department_id, designation, date_of_joining, basic_salary, allowances, deductions, employment_status)
VALUES
('EMP001','Arun Kumar','Male','1995-04-12','9876543210','arun.kumar@company.com','Chennai, TN',
 1,'HR Executive','2021-06-01', 32000.00, 4000.00, 1500.00, 'ACTIVE'),
('EMP002','Priya Sharma','Female','1997-09-23','9876501234','priya.sharma@company.com','Bengaluru, KA',
 3,'Software Engineer','2022-01-15', 55000.00, 8000.00, 3200.00, 'ACTIVE')
ON DUPLICATE KEY UPDATE name = name;
```

src/Employee.java

```
import java.math.BigDecimal;
import java.time.LocalDate;

/**
 * Employee.java
 * POJO model representing a single employee record.
 * Author: G Sai Deepak (192311432) | CSA0925 - Java Programming | CO5-AT2
 */
public class Employee {

    private String employeeId;
    private String name;
    private String gender;
    private LocalDate dateOfBirth;
    private String contactNumber;
    private String email;
    private String address;
    private int departmentId;
    private String departmentName; // used only for display (join result)
    private String designation;
    private LocalDate dateOfJoining;
    private BigDecimal basicSalary;
    private BigDecimal allowances;
    private BigDecimal deductions;
    private String employmentStatus;

    public Employee() { }

    public Employee(String employeeId, String name, String gender, LocalDate dateOfBirth,
                    String contactNumber, String email, String address, int departmentId,
                    String designation, LocalDate dateOfJoining, BigDecimal basicSalary,
                    BigDecimal allowances, BigDecimal deductions, String employmentStatus) {
        this.employeeId = employeeId;
        this.name = name;
        this.gender = gender;
        this.dateOfBirth = dateOfBirth;
        this.contactNumber = contactNumber;
        this.email = email;
        this.address = address;
        this.departmentId = departmentId;
        this.designation = designation;
        this.dateOfJoining = dateOfJoining;
        this.basicSalary = basicSalary;
        this.allowances = allowances;
        this.deductions = deductions;
        this.employmentStatus = employmentStatus;
    }

    // ---- Payroll calculations ----
    public BigDecimal getGrossSalary() {
        BigDecimal a = allowances == null ? BigDecimal.ZERO : allowances;
        return basicSalary.add(a);
    }

    public BigDecimal getNetSalary() {
        BigDecimal d = deductions == null ? BigDecimal.ZERO : deductions;
        return getGrossSalary().subtract(d);
    }

    // ---- Getters and setters ----
    public String getEmployeeId() { return employeeId; }
    public void setEmployeeId(String employeeId) { this.employeeId = employeeId; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getGender() { return gender; }
    public void setGender(String gender) { this.gender = gender; }

    public LocalDate getDateOfBirth() { return dateOfBirth; }
    public void setDateOfBirth(LocalDate dateOfBirth) { this.dateOfBirth = dateOfBirth; }

    public String getContactNumber() { return contactNumber; }
    public void setContactNumber(String contactNumber) { this.contactNumber = contactNumber; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getAddress() { return address; }
```

```
    public void setAddress(String address) { this.address = address; }

    public int getDepartmentId() { return departmentId; }
    public void setDepartmentId(int departmentId) { this.departmentId = departmentId; }

    public String getDepartmentName() { return departmentName; }
    public void setDepartmentName(String departmentName) { this.departmentName = departmentName; }

    public String getDesignation() { return designation; }
    public void setDesignation(String designation) { this.designation = designation; }

    public LocalDate getDateOfJoining() { return dateOfJoining; }
    public void setDateOfJoining(LocalDate dateOfJoining) { this.dateOfJoining = dateOfJoining; }

    public BigDecimal getBasicSalary() { return basicSalary; }
    public void setBasicSalary(BigDecimal basicSalary) { this.basicSalary = basicSalary; }

    public BigDecimal getAllowances() { return allowances; }
    public void setAllowances(BigDecimal allowances) { this.allowances = allowances; }

    public BigDecimal getDeductions() { return deductions; }
    public void setDeductions(BigDecimal deductions) { this.deductions = deductions; }

    public String getEmploymentStatus() { return employmentStatus; }
    public void setEmploymentStatus(String employmentStatus) { this.employmentStatus = employmentStatus; }
}
```

src/EmployeeValidationException.java

```
/**
 * EmployeeValidationException.java
 * Custom checked exception thrown when employee/payroll input data
 * fails business-rule validation (mandatory fields, numeric salary,
 * duplicate IDs, etc.).
 * Author: G Sai Deepak (192311432) | CSA0925 - Java Programming | CO5-AT2
 */
public class EmployeeValidationException extends Exception {
    public EmployeeValidationException(String message) {
        super(message);
    }
}
```

src/EmployeeNotFoundException.java

```
/**
 * EmployeeNotFoundException.java
 * Custom checked exception thrown when an update/delete/search
 * operation targets an Employee ID that does not exist in the database.
 * Author: G Sai Deepak (192311432) | CSA0925 - Java Programming | CO5-AT2
 */
public class EmployeeNotFoundException extends Exception {
    public EmployeeNotFoundException(String message) {
        super(message);
    }
}
```

src/DBConnectionUtil.java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * DBConnectionUtil.java
 * Centralised JDBC connection factory for the MySQL database
 * "employee_payroll_db". Update DB_URL / DB_USER / DB_PASSWORD
 * to match your local MySQL Workbench configuration.
 * Author: G Sai Deepak (192311432) | CSA0925 - Java Programming | CO5-AT2
 */
public class DBConnectionUtil {

    private static final String DB_URL =
            "jdbc:mysql://localhost:3306/employee_payroll_db?useSSL=false&serverTimezone=UTC";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "root";

    static {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            System.err.println("MySQL JDBC Driver not found on classpath: " + e.getMessage());
        }
    }

    /** Returns a fresh JDBC connection. Caller is responsible for closing it (try-with-resources). */
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
    }
}
```

src/EmployeeDAO.java

```
import  java.math.BigDecimal;
import java.sql.*;
import  java.time.LocalDate;
import  java.util.ArrayList;
import java.util.List;

/**
 * EmployeeDAO.java
 * Data-Access-Object encapsulating ALL database operations (CRUD) for the
 * Employee Management and Payroll application. Every operation uses
 * PreparedStatement (prevents SQL injection) and translates SQLExceptions
 * into meaningful, application-level errors for the GUI layer.
 * Author: G Sai Deepak (192311432) | CSA0925 - Java Programming | CO5-AT2
 */
public class EmployeeDAO {

    // ------------------------------------------------------------------------------------
    // CREATE
    // ------------------------------------------------------------------------------------
    public void insertEmployee(Employee emp) throws EmployeeValidationException, SQLException {
        validate(emp, true);

        String sql = "INSERT INTO employee (employee_id, name, gender, date_of_birth, "
                + "contact_number, email, address, department_id, designation, "
                + "date_of_joining, basic_salary, allowances, deductions, employment_status) "
                + "VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?)";

        try (Connection con = DBConnectionUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {

            if (employeeExists(con, emp.getEmployeeId())) {
                throw new EmployeeValidationException(
                        "Duplicate Employee ID: '" + emp.getEmployeeId() + "' already exists.");
            }

            bindEmployee(ps, emp);
            ps.executeUpdate();

        } catch (SQLIntegrityConstraintViolationException dup) {
            // e.g. duplicate email (UNIQUE) or invalid department_id (FK)
            throw new SQLException("Database constraint violated: " + dup.getMessage(), dup);
        }
    }

    // ------------------------------------------------------------------------------------
    // READ - single employee by ID
    // ------------------------------------------------------------------------------------
    public Employee getEmployeeById(String employeeId) throws EmployeeNotFoundException, SQLException {
        String sql = "SELECT e.*, d.department_name FROM employee e "
                + "JOIN department d ON e.department_id = d.department_id "
                + "WHERE e.employee_id = ?";

        try (Connection con = DBConnectionUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, employeeId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return mapRow(rs);
                }
            }
        }
        throw new EmployeeNotFoundException("No employee found with ID: " + employeeId);
    }

    // ------------------------------------------------------------------------------------
    // READ - search by department / designation / status (any may be null/blank)
    // ------------------------------------------------------------------------------------
    public List<Employee> searchEmployees(String department, String designation, String status)
            throws SQLException {

        StringBuilder sql = new StringBuilder(
                "SELECT e.*, d.department_name FROM employee e "
                        + "JOIN department d ON e.department_id = d.department_id WHERE 1=1 ");
        List<Object> params = new ArrayList<>();

        if (department != null && !department.isBlank()) {
            sql.append("AND d.department_name = ? ");
            params.add(department);
```

```
    }
    if (designation != null && !designation.isBlank()) {
        sql.append("AND e.designation LIKE ? ");
        params.add("%" + designation + "%");
    }
    if (status != null && !status.isBlank()) {
        sql.append("AND e.employment_status = ? ");
        params.add(status);
    }
    sql.append("ORDER BY e.employee_id");

    List<Employee> results = new ArrayList<>();
    try (Connection con = DBConnectionUtil.getConnection();
         PreparedStatement ps = con.prepareStatement(sql.toString())) {
        for (int i = 0; i < params.size(); i++) {
            ps.setObject(i + 1, params.get(i));
        }
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                results.add(mapRow(rs));
            }
        }
    }
    return results;
}

// ------------------------------------------------------------------------------------
// READ - all employees (for the list/view screen)
// ------------------------------------------------------------------------------------
public List<Employee> getAllEmployees() throws SQLException {
    return searchEmployees(null, null, null);
}

// ------------------------------------------------------------------------------------
// UPDATE
// ------------------------------------------------------------------------------------
public void updateEmployee(Employee emp) throws EmployeeValidationException,
        EmployeeNotFoundException, SQLException {

    validate(emp, false);

    String sql = "UPDATE employee SET name=?, gender=?, date_of_birth=?, contact_number=?, "
            + "email=?, address=?, department_id=?, designation=?, date_of_joining=?, "
            + "basic_salary=?, allowances=?, deductions=?, employment_status=? "
            + "WHERE employee_id=?";

    try (Connection con = DBConnectionUtil.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, emp.getName());
        ps.setString(2, emp.getGender());
        ps.setDate(3, emp.getDateOfBirth() != null ? Date.valueOf(emp.getDateOfBirth()) : null);
        ps.setString(4, emp.getContactNumber());
        ps.setString(5, emp.getEmail());
        ps.setString(6, emp.getAddress());
        ps.setInt(7, emp.getDepartmentId());
        ps.setString(8, emp.getDesignation());
        ps.setDate(9, Date.valueOf(emp.getDateOfJoining()));
        ps.setBigDecimal(10, emp.getBasicSalary());
        ps.setBigDecimal(11, emp.getAllowances());
        ps.setBigDecimal(12, emp.getDeductions());
        ps.setString(13, emp.getEmploymentStatus());
        ps.setString(14, emp.getEmployeeId());

        int rows = ps.executeUpdate();
        if (rows == 0) {
            throw new EmployeeNotFoundException(
                    "Update failed - no employee exists with ID: " + emp.getEmployeeId());
        }
    }
}

// ------------------------------------------------------------------------------------
// DELETE
// ------------------------------------------------------------------------------------
public void deleteEmployee(String employeeId) throws EmployeeNotFoundException, SQLException {
    String sql = "DELETE FROM employee WHERE employee_id = ?";
    try (Connection con = DBConnectionUtil.getConnection();
         PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, employeeId);
```

```
        int rows = ps.executeUpdate();
        if (rows == 0) {
            throw new EmployeeNotFoundException(
                    "Delete failed - no employee exists with ID: " + employeeId);
        }
    }
}

// ------------------------------------------------------------------------------------
// READ - departments (for populating the Choice/dropdown in the GUI)
// ------------------------------------------------------------------------------------
public java.util.LinkedHashMap<Integer, String> getDepartments() throws SQLException {
    java.util.LinkedHashMap<Integer, String> map = new java.util.LinkedHashMap<>();
    String sql = "SELECT department_id, department_name FROM department ORDER BY department_id";
    try (Connection con = DBConnectionUtil.getConnection();
         PreparedStatement ps = con.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            map.put(rs.getInt("department_id"),  rs.getString("department_name"));
        }
    }
    return map;
}

// ------------------------------------------------------------------------------------
// Helper: does employee_id already exist?
// ------------------------------------------------------------------------------------
private boolean employeeExists(Connection con, String employeeId) throws SQLException {
    String sql = "SELECT 1 FROM employee WHERE employee_id = ?";
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setString(1, employeeId);
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next();
        }
    }
}

private void bindEmployee(PreparedStatement ps, Employee emp) throws SQLException {
    ps.setString(1, emp.getEmployeeId());
    ps.setString(2, emp.getName());
    ps.setString(3, emp.getGender());
    ps.setDate(4, emp.getDateOfBirth() != null ? Date.valueOf(emp.getDateOfBirth()) : null);
    ps.setString(5, emp.getContactNumber());
    ps.setString(6, emp.getEmail());
    ps.setString(7, emp.getAddress());
    ps.setInt(8, emp.getDepartmentId());
    ps.setString(9, emp.getDesignation());
    ps.setDate(10, Date.valueOf(emp.getDateOfJoining()));
    ps.setBigDecimal(11, emp.getBasicSalary());
    ps.setBigDecimal(12, emp.getAllowances());
    ps.setBigDecimal(13, emp.getDeductions());
    ps.setString(14, emp.getEmploymentStatus());
}

private Employee mapRow(ResultSet rs) throws SQLException {
    Employee emp = new Employee();
    emp.setEmployeeId(rs.getString("employee_id"));
    emp.setName(rs.getString("name"));
    emp.setGender(rs.getString("gender"));
    Date dob = rs.getDate("date_of_birth");
    emp.setDateOfBirth(dob != null ? dob.toLocalDate() : null);
    emp.setContactNumber(rs.getString("contact_number"));
    emp.setEmail(rs.getString("email"));
    emp.setAddress(rs.getString("address"));
    emp.setDepartmentId(rs.getInt("department_id"));
    emp.setDepartmentName(rs.getString("department_name"));
    emp.setDesignation(rs.getString("designation"));
    Date doj = rs.getDate("date_of_joining");
    emp.setDateOfJoining(doj != null ? doj.toLocalDate() : null);
    emp.setBasicSalary(rs.getBigDecimal("basic_salary"));
    emp.setAllowances(rs.getBigDecimal("allowances"));
    emp.setDeductions(rs.getBigDecimal("deductions"));
    emp.setEmploymentStatus(rs.getString("employment_status"));
    return emp;
}

// ------------------------------------------------------------------------------------
// Business-rule validation (mandatory fields, numeric salary, dates)
// ------------------------------------------------------------------------------------
private void validate(Employee emp, boolean isInsert) throws EmployeeValidationException {
```

```
        if (isBlank(emp.getEmployeeId())) {
            throw new EmployeeValidationException("Employee ID is mandatory.");
        }
        if (isBlank(emp.getName())) {
            throw new EmployeeValidationException("Employee name is mandatory.");
        }
        if (isBlank(emp.getContactNumber()) || !emp.getContactNumber().matches("\\d{10}")) {
            throw new EmployeeValidationException("Contact number must be a valid 10-digit number.");
        }
        if (isBlank(emp.getEmail()) || !emp.getEmail().matches("^[\\w.+-]+@[\\w-]+\\.[a-zA-Z]{2,}$")) {
            throw new EmployeeValidationException("A valid email address is mandatory.");
        }
        if (emp.getDepartmentId() <= 0) {
            throw new EmployeeValidationException("Department must be selected.");
        }
        if (emp.getDateOfJoining() == null) {
            throw new EmployeeValidationException("Date of joining is mandatory.");
        }
        if (emp.getDateOfJoining().isAfter(LocalDate.now())) {
            throw new EmployeeValidationException("Date of joining cannot be in the future.");
        }
        validatePositiveDecimal(emp.getBasicSalary(), "Basic salary");
        validateNonNegativeDecimal(emp.getAllowances(), "Allowances");
        validateNonNegativeDecimal(emp.getDeductions(), "Deductions");

        if (emp.getEmploymentStatus() == null
                || !emp.getEmploymentStatus().matches("ACTIVE|INACTIVE|ON_LEAVE|TERMINATED")) {
            throw new EmployeeValidationException("Employment status is invalid.");
        }
    }

    private void validatePositiveDecimal(BigDecimal value, String field) throws EmployeeValidationException {
        if (value == null) {
            throw new EmployeeValidationException(field + " is mandatory and must be numeric.");
        }
        if (value.compareTo(BigDecimal.ZERO) < 0) {
            throw new EmployeeValidationException(field + " cannot be negative.");
        }
    }

    private void validateNonNegativeDecimal(BigDecimal value, String field) throws EmployeeValidationException {
        if (value != null && value.compareTo(BigDecimal.ZERO) < 0) {
            throw new EmployeeValidationException(field + " cannot be negative.");
        }
    }

    private boolean isBlank(String s) {
        return s == null || s.trim().isEmpty();
    }
}
```

src/EmployeeManagementApp.java

```
import java.awt.*;
import java.awt.event.*;
import java.math.BigDecimal;
import java.sql.SQLException;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.LinkedHashMap;
import java.util.List;

/**
 * EmployeeManagementApp.java
 * ---------------------------------------------------------------------------------------
 * CO5-AT2 : Employee Management and Payroll Application (Java AWT + JDBC + MySQL)
 * Subject : CSA0925 - Java Programming
 * Student : G Sai Deepak    Reg. No: 192311432
 * ---------------------------------------------------------------------------------------
 * A single-frame, CardLayout based desktop application offering:
 *   1. Employee Registration
 *   2. Employee Search
 *   3. Employee Update
 *   4. Employee Delete
 *   5. Employee List / View
 *   6. Payroll / Salary Calculation
 *
 * All database access goes through EmployeeDAO (JDBC, PreparedStatement).
 * All user input is validated before it reaches the database, and every
 * database/validation error is caught and shown to the user via a Dialog
 * instead of crashing the application.
 * ---------------------------------------------------------------------------------------
 */
public class EmployeeManagementApp extends Frame implements ActionListener {

    private static final DateTimeFormatter DATE_FMT = DateTimeFormatter.ofPattern("dd-MM-yyyy");

    private final EmployeeDAO dao = new EmployeeDAO();
    private final CardLayout cardLayout = new CardLayout();
    private final Panel cards = new Panel(cardLayout);

    // ---- Registration screen fields ----
    private TextField regId, regName, regContact, regEmail, regDesignation,
            regDoj, regDob, regBasic, regAllow, regDeduct;
    private TextArea regAddress;
    private Choice regGender, regDept, regStatus;

    // ---- Search screen fields  ----
    private Choice searchDept, searchStatus;
    private TextField searchDesignation, searchById;
    private TextArea searchResults;

    // ---- Update screen fields ----
    private TextField updLoadId, updName, updContact, updEmail, updDesignation,
            updDoj, updDob, updBasic, updAllow, updDeduct;
    private TextArea updAddress;
    private Choice updGender, updDept, updStatus;
    private String updCurrentId = null;

    // ---- Delete screen fields ----
    private TextField delId;
    private TextArea delPreview;

    // ---- List/View screen ----
    private TextArea listArea;

    // ---- Payroll screen ----
    private TextField payId;
    private TextArea payResult;

    public EmployeeManagementApp() {
        super("Employee Management and Payroll Application  |  CSA0925 - Java Programming (CO5-AT2)  |  G Sai Deepak -
        setLayout(new BorderLayout());
        add(buildHeader(), BorderLayout.NORTH);
        add(cards, BorderLayout.CENTER);

        cards.add(buildMenuPanel(), "MENU");
        cards.add(buildRegistrationPanel(), "REGISTER");
        cards.add(buildSearchPanel(), "SEARCH");
        cards.add(buildUpdatePanel(), "UPDATE");
```

```
        cards.add(buildDeletePanel(), "DELETE");
        cards.add(buildListPanel(), "LIST");
        cards.add(buildPayrollPanel(), "PAYROLL");

        cardLayout.show(cards, "MENU");

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                dispose();
                System.exit(0);
            }
        });

        setSize(900, 650);
        setLocationRelativeTo(null);
        setVisible(true);
    }

    private Panel buildHeader() {
        Panel header = new Panel(new FlowLayout(FlowLayout.CENTER));
        Label title = new Label("Employee Management and Payroll System");
        title.setFont(new Font("SansSerif", Font.BOLD, 18));
        header.add(title);
        return header;
    }

    // ================================================================
    // MENU
    // ================================================================
    private Panel buildMenuPanel() {
        Panel p = new Panel(new GridLayout(7, 1, 10, 10));
        p.add(navButton("1. Employee Registration", "REGISTER"));
        p.add(navButton("2. Employee Search", "SEARCH"));
        p.add(navButton("3. Employee Update", "UPDATE"));
        p.add(navButton("4. Employee Delete", "DELETE"));
        p.add(navButton("5. Employee List / View", "LIST"));
        p.add(navButton("6. Payroll / Salary Calculation", "PAYROLL"));
        Button exit = new Button("Exit");
        exit.addActionListener(e -> { dispose(); System.exit(0); });
        p.add(exit);
        Panel wrap = new Panel(new FlowLayout(FlowLayout.CENTER));
        wrap.add(p);
        return wrap;
    }

    private Button navButton(String label, String card) {
        Button b = new Button(label);
        b.addActionListener(e -> { refreshCardIfNeeded(card); cardLayout.show(cards, card); });
        return b;
    }

    private void refreshCardIfNeeded(String card) {
        try {
            if (card.equals("LIST")) refreshList();
            if (card.equals("REGISTER")) reloadDepartments(regDept);
            if (card.equals("SEARCH")) reloadDepartments(searchDept);
            if (card.equals("UPDATE")) reloadDepartments(updDept);
        } catch (SQLException ex) {
            showError("Could not load data from database: " + ex.getMessage());
        }
    }

    private Button backButton() {
        Button back = new Button("<< Back to Menu");
        back.addActionListener(e -> cardLayout.show(cards, "MENU"));
        return back;
    }

    // ================================================================
    // 1. EMPLOYEE REGISTRATION
    // ================================================================
    private Panel buildRegistrationPanel() {
        Panel outer = new Panel(new BorderLayout());
        Panel form = new Panel(new GridLayout(0, 2, 6, 6));

        regId = new TextField();
        regName = new TextField();
        regGender = new Choice();
        regGender.add("Male"); regGender.add("Female"); regGender.add("Other");
        regDob = new TextField("dd-mm-yyyy");
```

```
    regContact = new TextField();
    regEmail = new TextField();
    regAddress = new TextArea("", 3, 20, TextArea.SCROLLBARS_VERTICAL_ONLY);
    regDept = new Choice();
    regDesignation = new TextField();
    regDoj = new TextField("dd-mm-yyyy");
    regBasic = new TextField();
    regAllow = new TextField("0");
    regDeduct = new TextField("0");
    regStatus = new Choice();
    regStatus.add("ACTIVE"); regStatus.add("INACTIVE"); regStatus.add("ON_LEAVE"); regStatus.add("TERMINATED");

    addFormRow(form, "Employee ID *:", regId);
    addFormRow(form, "Name *:", regName);
    addFormRow(form, "Gender:", regGender);
    addFormRow(form, "Date of Birth (dd-mm-yyyy):", regDob);
    addFormRow(form, "Contact Number * (10 digits):", regContact);
    addFormRow(form, "Email *:", regEmail);
    addFormRow(form, "Address:", regAddress);
    addFormRow(form, "Department *:", regDept);
    addFormRow(form, "Designation:", regDesignation);
    addFormRow(form, "Date of Joining * (dd-mm-yyyy):", regDoj);
    addFormRow(form, "Basic Salary *:", regBasic);
    addFormRow(form, "Allowances:", regAllow);
    addFormRow(form, "Deductions:", regDeduct);
    addFormRow(form, "Employment Status:", regStatus);

    Button submit = new Button("Register Employee");
    submit.addActionListener(this);
    submit.setActionCommand("REG_SUBMIT");
    Button clear = new Button("Clear");
    clear.addActionListener(e -> clearRegistrationForm());

    Panel btns = new Panel(new FlowLayout());
    btns.add(submit); btns.add(clear); btns.add(backButton());

    outer.add(new Label("Employee Registration", Label.CENTER), BorderLayout.NORTH);
    outer.add(wrapScroll(form), BorderLayout.CENTER);
    outer.add(btns, BorderLayout.SOUTH);
    return outer;
}

private void clearRegistrationForm() {
    regId.setText(""); regName.setText(""); regDob.setText("dd-mm-yyyy");
    regContact.setText(""); regEmail.setText(""); regAddress.setText("");
    regDesignation.setText(""); regDoj.setText("dd-mm-yyyy");
    regBasic.setText(""); regAllow.setText("0"); regDeduct.setText("0");
}

private void handleRegisterSubmit() {
    try {
        Employee emp = new Employee();
        emp.setEmployeeId(regId.getText().trim());
        emp.setName(regName.getText().trim());
        emp.setGender(regGender.getSelectedItem());
        emp.setDateOfBirth(parseDateOptional(regDob.getText().trim()));
        emp.setContactNumber(regContact.getText().trim());
        emp.setEmail(regEmail.getText().trim());
        emp.setAddress(regAddress.getText().trim());
        emp.setDepartmentId(selectedDeptId(regDept));
        emp.setDesignation(regDesignation.getText().trim());
        emp.setDateOfJoining(parseDateRequired(regDoj.getText().trim(), "Date of joining"));
        emp.setBasicSalary(parseAmount(regBasic.getText().trim(), "Basic salary"));
        emp.setAllowances(parseAmount(regAllow.getText().trim().isEmpty() ? "0" : regAllow.getText().trim(), "Allo
        emp.setDeductions(parseAmount(regDeduct.getText().trim().isEmpty() ? "0" : regDeduct.getText().trim(), "De
        emp.setEmploymentStatus(regStatus.getSelectedItem());

        dao.insertEmployee(emp);
        showInfo("Employee '" + emp.getEmployeeId() + "' registered successfully.");
        clearRegistrationForm();

    } catch (EmployeeValidationException ve) {
        showError("Validation error: " + ve.getMessage());
    } catch (SQLException se) {
        showError("Database error while registering employee: " + se.getMessage());
    } catch (NumberFormatException | DateTimeParseException fe) {
        showError("Invalid input format: " + fe.getMessage());
    }
}
// ============================================================
```

```
// 2. EMPLOYEE SEARCH
// ================================================================
private Panel buildSearchPanel() {
    Panel outer = new Panel(new BorderLayout());

    Panel filterRow = new Panel(new FlowLayout(FlowLayout.LEFT));
    searchDept = new Choice(); searchDept.add(""); // blank = any
    searchStatus = new Choice();
    searchStatus.add(""); searchStatus.add("ACTIVE"); searchStatus.add("INACTIVE");
    searchStatus.add("ON_LEAVE"); searchStatus.add("TERMINATED");
    searchDesignation = new TextField(10);

    filterRow.add(new Label("Department:")); filterRow.add(searchDept);
    filterRow.add(new Label("Designation contains:")); filterRow.add(searchDesignation);
    filterRow.add(new Label("Status:")); filterRow.add(searchStatus);
    Button searchBtn = new Button("Search");
    searchBtn.addActionListener(e -> handleSearch());
    filterRow.add(searchBtn);

    Panel idRow = new Panel(new FlowLayout(FlowLayout.LEFT));
    searchById = new TextField(10);
    Button byIdBtn = new Button("Find by Employee ID");
    byIdBtn.addActionListener(e -> handleSearchById());
    idRow.add(new Label("Employee ID:")); idRow.add(searchById); idRow.add(byIdBtn);

    Panel top = new Panel(new GridLayout(2, 1));
    top.add(filterRow); top.add(idRow);

    searchResults = new TextArea("", 20, 90, TextArea.SCROLLBARS_VERTICAL_ONLY);
    searchResults.setEditable(false);
    searchResults.setFont(new Font("Monospaced", Font.PLAIN, 12));

    outer.add(new Label("Employee Search", Label.CENTER), BorderLayout.NORTH);
    outer.add(top, BorderLayout.NORTH);
    outer.add(searchResults, BorderLayout.CENTER);
    outer.add(backButton(), BorderLayout.SOUTH);
    return outer;
}

private void handleSearch() {
    try {
        List<Employee> results = dao.searchEmployees(
                searchDept.getSelectedItem(), searchDesignation.getText().trim(),
                searchStatus.getSelectedItem());
        searchResults.setText(formatTable(results));
        if (results.isEmpty()) showInfo("No employees matched the given search criteria.");
    } catch (SQLException se) {
        showError("Database error during search: " + se.getMessage());
    }
}

private void handleSearchById() {
    String id = searchById.getText().trim();
    if (id.isEmpty()) { showError("Please enter an Employee ID to search."); return; }
    try {
        Employee emp = dao.getEmployeeById(id);
        searchResults.setText(formatTable(List.of(emp)) + "\n\n" + formatDetail(emp));
    } catch (EmployeeNotFoundException nf) {
        searchResults.setText("");
        showError(nf.getMessage());
    } catch (SQLException se) {
        showError("Database error while searching by ID: " + se.getMessage());
    }
}

// ================================================================
// 3. EMPLOYEE UPDATE
// ================================================================
private Panel buildUpdatePanel() {
    Panel outer = new Panel(new BorderLayout());

    Panel loadRow = new Panel(new FlowLayout(FlowLayout.LEFT));
    updLoadId = new TextField(10);
    Button loadBtn = new Button("Load Employee");
    loadBtn.addActionListener(e -> handleLoadForUpdate());
    loadRow.add(new Label("Employee ID:")); loadRow.add(updLoadId); loadRow.add(loadBtn);

    Panel form = new Panel(new GridLayout(0, 2, 6, 6));
    updName = new TextField();
    updGender = new Choice();
```

```
        updGender.add("Male"); updGender.add("Female"); updGender.add("Other");
        updDob = new TextField();
        updContact = new TextField();
        updEmail = new TextField();
        updAddress = new TextArea("", 3, 20, TextArea.SCROLLBARS_VERTICAL_ONLY);
        updDept = new Choice();
        updDesignation = new TextField();
        updDoj = new TextField();
        updBasic = new TextField();
        updAllow = new TextField();
        updDeduct = new TextField();
        updStatus = new Choice();
        updStatus.add("ACTIVE"); updStatus.add("INACTIVE"); updStatus.add("ON_LEAVE"); updStatus.add("TERMINATED");

        addFormRow(form, "Name *:", updName);
        addFormRow(form, "Gender:", updGender);
        addFormRow(form, "Date of Birth (dd-mm-yyyy):", updDob);
        addFormRow(form, "Contact Number * (10 digits):", updContact);
        addFormRow(form, "Email *:", updEmail);
        addFormRow(form, "Address:", updAddress);
        addFormRow(form, "Department *:", updDept);
        addFormRow(form, "Designation:", updDesignation);
        addFormRow(form, "Date of Joining * (dd-mm-yyyy):", updDoj);
        addFormRow(form, "Basic Salary *:", updBasic);
        addFormRow(form, "Allowances:", updAllow);
        addFormRow(form, "Deductions:", updDeduct);
        addFormRow(form, "Employment Status:", updStatus);

        Button save = new Button("Save Changes");
        save.addActionListener(e -> handleUpdateSave());
        Panel btns = new Panel(new FlowLayout());
        btns.add(save); btns.add(backButton());

        outer.add(new Label("Employee Update", Label.CENTER), BorderLayout.NORTH);
        Panel center = new Panel(new BorderLayout());
        center.add(loadRow, BorderLayout.NORTH);
        center.add(wrapScroll(form), BorderLayout.CENTER);
        outer.add(center, BorderLayout.CENTER);
        outer.add(btns, BorderLayout.SOUTH);
        return outer;
    }

    private void handleLoadForUpdate() {
        String id = updLoadId.getText().trim();
        if (id.isEmpty()) { showError("Enter an Employee ID to load."); return; }
        try {
            Employee emp = dao.getEmployeeById(id);
            updCurrentId = emp.getEmployeeId();
            updName.setText(emp.getName());
            selectChoiceSafe(updGender, emp.getGender());
            updDob.setText(emp.getDateOfBirth() != null ? emp.getDateOfBirth().format(DATE_FMT) : "");
            updContact.setText(emp.getContactNumber());
            updEmail.setText(emp.getEmail());
            updAddress.setText(emp.getAddress() == null ? "" : emp.getAddress());
            selectChoiceSafe(updDept, emp.getDepartmentName());
            updDesignation.setText(emp.getDesignation() == null ? "" : emp.getDesignation());
            updDoj.setText(emp.getDateOfJoining().format(DATE_FMT));
            updBasic.setText(emp.getBasicSalary().toPlainString());
            updAllow.setText(emp.getAllowances() == null ? "0" : emp.getAllowances().toPlainString());
            updDeduct.setText(emp.getDeductions() == null ? "0" : emp.getDeductions().toPlainString());
            selectChoiceSafe(updStatus, emp.getEmploymentStatus());
            showInfo("Employee '" + id + "' loaded. Edit fields and click 'Save Changes'.");
        } catch (EmployeeNotFoundException nf) {
            updCurrentId = null;
            showError(nf.getMessage());
        } catch (SQLException se) {
            showError("Database error while loading employee: " + se.getMessage());
        }
    }

    private void handleUpdateSave() {
        if (updCurrentId == null) {
            showError("Load an existing employee first (enter ID and click 'Load Employee').");
            return;
        }
        try {
            Employee emp = new Employee();
            emp.setEmployeeId(updCurrentId);
            emp.setName(updName.getText().trim());
            emp.setGender(updGender.getSelectedItem());
```

```
            emp.setDateOfBirth(parseDateOptional(updDob.getText().trim()));
            emp.setContactNumber(updContact.getText().trim());
            emp.setEmail(updEmail.getText().trim());
            emp.setAddress(updAddress.getText().trim());
            emp.setDepartmentId(selectedDeptId(updDept));
            emp.setDesignation(updDesignation.getText().trim());
            emp.setDateOfJoining(parseDateRequired(updDoj.getText().trim(), "Date of joining"));
            emp.setBasicSalary(parseAmount(updBasic.getText().trim(), "Basic salary"));
            emp.setAllowances(parseAmount(updAllow.getText().trim(), "Allowances"));
            emp.setDeductions(parseAmount(updDeduct.getText().trim(), "Deductions"));
            emp.setEmploymentStatus(updStatus.getSelectedItem());

            dao.updateEmployee(emp);
            showInfo("Employee '" + updCurrentId + "' updated successfully.");
        } catch (EmployeeValidationException ve) {
            showError("Validation error: " + ve.getMessage());
        } catch (EmployeeNotFoundException nf) {
            showError(nf.getMessage());
        } catch (SQLException se) {
            showError("Database error while updating employee: " + se.getMessage());
        } catch (NumberFormatException | DateTimeParseException fe) {
            showError("Invalid input format: " + fe.getMessage());
        }
    }

    // ================================================================
    // 4. EMPLOYEE DELETE
    // ================================================================
    private Panel buildDeletePanel() {
        Panel outer = new Panel(new BorderLayout());
        Panel row = new Panel(new FlowLayout(FlowLayout.LEFT));
        delId = new TextField(10);
        Button preview = new Button("Preview");
        preview.addActionListener(e -> handleDeletePreview());
        Button delete = new Button("Delete Employee");
        delete.addActionListener(e -> handleDeleteConfirm());
        row.add(new Label("Employee ID:")); row.add(delId); row.add(preview); row.add(delete);

        delPreview = new TextArea("", 15, 90, TextArea.SCROLLBARS_VERTICAL_ONLY);
        delPreview.setEditable(false);

        outer.add(new Label("Employee Delete", Label.CENTER), BorderLayout.NORTH);
        outer.add(row, BorderLayout.NORTH);
        outer.add(delPreview, BorderLayout.CENTER);
        outer.add(backButton(), BorderLayout.SOUTH);
        return outer;
    }

    private void handleDeletePreview() {
        String id = delId.getText().trim();
        if (id.isEmpty()) { showError("Enter an Employee ID."); return; }
        try {
            Employee emp = dao.getEmployeeById(id);
            delPreview.setText(formatDetail(emp));
        } catch (EmployeeNotFoundException nf) {
            delPreview.setText("");
            showError(nf.getMessage());
        } catch (SQLException se) {
            showError("Database error: " + se.getMessage());
        }
    }

    private void handleDeleteConfirm() {
        String id = delId.getText().trim();
        if (id.isEmpty()) { showError("Enter an Employee ID."); return; }

        Dialog confirm = new Dialog(this, "Confirm Delete", true);
        confirm.setLayout(new BorderLayout());
        confirm.add(new Label("  Are you sure you want to delete employee '" + id + "'?  ", Label.CENTER),
                BorderLayout.CENTER);
        Panel btns = new Panel(new FlowLayout(FlowLayout.CENTER));
        Button yes = new Button("Yes, Delete");
        Button no = new Button("Cancel");
        yes.addActionListener(e -> {
            confirm.dispose();
            try {
                dao.deleteEmployee(id);
                showInfo("Employee '" + id + "' deleted successfully.");
                delPreview.setText("");
                delId.setText("");
```

```
        } catch (EmployeeNotFoundException nf) {
            showError(nf.getMessage());
        } catch (SQLException se) {
            showError("Database error while deleting employee: " + se.getMessage());
        }
    });
    no.addActionListener(e -> confirm.dispose());
    btns.add(yes); btns.add(no);
    confirm.add(btns, BorderLayout.SOUTH);
    confirm.setSize(360, 130);
    confirm.setLocationRelativeTo(this);
    confirm.setVisible(true);
}

// ================================================================
// 5. EMPLOYEE LIST / VIEW
// ================================================================
private Panel buildListPanel() {
    Panel outer = new Panel(new BorderLayout());
    listArea = new TextArea("", 25, 100, TextArea.SCROLLBARS_VERTICAL_ONLY);
    listArea.setEditable(false);
    listArea.setFont(new Font("Monospaced", Font.PLAIN, 12));

    Button refresh = new Button("Refresh List");
    refresh.addActionListener(e -> refreshList());

    Panel btns = new Panel(new FlowLayout());
    btns.add(refresh); btns.add(backButton());

    outer.add(new Label("Employee List / View", Label.CENTER), BorderLayout.NORTH);
    outer.add(listArea, BorderLayout.CENTER);
    outer.add(btns, BorderLayout.SOUTH);
    return outer;
}

private void refreshList() {
    try {
        List<Employee> all = dao.getAllEmployees();
        listArea.setText(formatTable(all));
    } catch (SQLException se) {
        showError("Database error while loading employee list: " + se.getMessage());
    }
}

// ================================================================
// 6. PAYROLL / SALARY CALCULATION
// ================================================================
private Panel buildPayrollPanel() {
    Panel outer = new Panel(new BorderLayout());
    Panel row = new Panel(new FlowLayout(FlowLayout.LEFT));
    payId = new TextField(10);
    Button calc = new Button("Calculate Payroll");
    calc.addActionListener(e -> handlePayrollForOne());
    Button calcAll = new Button("Payroll for All Employees");
    calcAll.addActionListener(e -> handlePayrollForAll());
    row.add(new Label("Employee ID:")); row.add(payId); row.add(calc); row.add(calcAll);

    payResult = new TextArea("", 20, 90, TextArea.SCROLLBARS_VERTICAL_ONLY);
    payResult.setEditable(false);
    payResult.setFont(new Font("Monospaced", Font.PLAIN, 12));

    outer.add(new Label("Payroll / Salary Calculation", Label.CENTER), BorderLayout.NORTH);
    outer.add(row, BorderLayout.NORTH);
    outer.add(payResult, BorderLayout.CENTER);
    outer.add(backButton(), BorderLayout.SOUTH);
    return outer;
}

private void handlePayrollForOne() {
    String id = payId.getText().trim();
    if (id.isEmpty()) { showError("Enter an Employee ID."); return; }
    try {
        Employee emp = dao.getEmployeeById(id);
        payResult.setText(formatPayrollHeader() + formatPayrollRow(emp));
    } catch (EmployeeNotFoundException nf) {
        payResult.setText("");
        showError(nf.getMessage());
    } catch (SQLException se) {
        showError("Database error while calculating payroll: " + se.getMessage());
    }
```

```
}

private void handlePayrollForAll() {
    try {
        List<Employee> all = dao.getAllEmployees();
        StringBuilder sb = new StringBuilder(formatPayrollHeader());
        for (Employee emp : all) sb.append(formatPayrollRow(emp));
        payResult.setText(sb.toString());
    } catch (SQLException se) {
        showError("Database error while calculating payroll: " + se.getMessage());
    }
}

private String formatPayrollHeader() {
    return String.format("%-10s %-20s %12s %12s %12s %12s%n", "ID", "Name", "Basic", "Allowances", "Deductions", "
            + "-".repeat(85) + "\n";
}

private String formatPayrollRow(Employee e) {
    return String.format("%-10s %-20s %12s %12s %12s %12s%n",
            e.getEmployeeId(), e.getName(),
            e.getBasicSalary(), e.getAllowances(), e.getDeductions(), e.getNetSalary());
}

// ================================================================
// Shared helpers
// ================================================================
private void addFormRow(Panel form, String label, Component field) {
    form.add(new Label(label));
    form.add(field);
}

private Panel wrapScroll(Panel form) {
    ScrollPane sp = new ScrollPane();
    sp.add(form);
    Panel wrap = new Panel(new BorderLayout());
    wrap.add(sp, BorderLayout.CENTER);
    return wrap;
}

private void reloadDepartments(Choice choice) throws SQLException {
    LinkedHashMap<Integer, String> depts = dao.getDepartments();
    choice.removeAll();
    if (choice == searchDept) choice.add("");
    for (String name : depts.values()) choice.add(name);
}

private int selectedDeptId(Choice choice) throws SQLException {
    String selected = choice.getSelectedItem();
    LinkedHashMap<Integer, String> depts = dao.getDepartments();
    for (var entry : depts.entrySet()) {
        if (entry.getValue().equals(selected)) return entry.getKey();
    }
    return -1;
}

private void selectChoiceSafe(Choice choice, String value) {
    if (value == null) return;
    for (int i = 0; i < choice.getItemCount(); i++) {
        if (choice.getItem(i).equalsIgnoreCase(value)) { choice.select(i); return; }
    }
}

private LocalDate parseDateRequired(String text, String fieldName) throws EmployeeValidationException {
    if (text == null || text.isBlank()) {
        throw new EmployeeValidationException(fieldName + " is mandatory (format dd-mm-yyyy).");
    }
    try {
        return LocalDate.parse(text, DATE_FMT);
    } catch (DateTimeParseException ex) {
        throw new EmployeeValidationException(fieldName + " must be a valid date in dd-mm-yyyy format.");
    }
}

private LocalDate parseDateOptional(String text) {
    if (text == null || text.isBlank() || text.equals("dd-mm-yyyy")) return null;
    try {
        return LocalDate.parse(text, DATE_FMT);
    } catch (DateTimeParseException ex) {
        return null;
```

```
        }
    }

    private BigDecimal parseAmount(String text, String fieldName) throws EmployeeValidationException {
        try {
            BigDecimal val = new BigDecimal(text);
            if (val.compareTo(BigDecimal.ZERO) < 0) {
                throw new EmployeeValidationException(fieldName + " cannot be negative.");
            }
            return val;
        } catch (NumberFormatException nfe) {
            throw new EmployeeValidationException(fieldName + " must be a valid numeric value.");
        }
    }

    private String formatTable(List<Employee> list) {
        if (list.isEmpty()) return "(no records found)";
        StringBuilder sb = new StringBuilder();
        sb.append(String.format("%-10s %-18s %-12s %-14s %-10s %-10s%n",
                "ID", "Name", "Department", "Designation", "Status", "Net Salary"));
        sb.append("-".repeat(85)).append("\n");
        for (Employee e : list) {
            sb.append(String.format("%-10s %-18s %-12s %-14s %-10s %-10s%n",
                    e.getEmployeeId(), e.getName(), e.getDepartmentName(),
                    e.getDesignation() == null ? "" : e.getDesignation(),
                    e.getEmploymentStatus(), e.getNetSalary()));
        }
        return sb.toString();
    }

    private String formatDetail(Employee e) {
        return "Employee ID       : " + e.getEmployeeId() + "\n"
                + "Name              : " + e.getName() + "\n"
                + "Gender            : " + e.getGender() + "\n"
                + "Date of Birth     : " + (e.getDateOfBirth() != null ? e.getDateOfBirth().format(DATE_FMT) : "-") +
                + "Contact Number    : " + e.getContactNumber() + "\n"
                + "Email             : " + e.getEmail() + "\n"
                + "Address           : " + (e.getAddress() == null ? "-" : e.getAddress()) + "\n"
                + "Department        : " + e.getDepartmentName() + "\n"
                + "Designation       : " + (e.getDesignation() == null ? "-" : e.getDesignation()) + "\n"
                + "Date of Joining   : " + e.getDateOfJoining().format(DATE_FMT) + "\n"
                + "Basic Salary      : " + e.getBasicSalary() + "\n"
                + "Allowances        : " + e.getAllowances() + "\n"
                + "Deductions        : " + e.getDeductions() + "\n"
                + "Gross Salary      : " + e.getGrossSalary() + "\n"
                + "Net Salary        : " + e.getNetSalary() + "\n"
                + "Employment Status : " + e.getEmploymentStatus();
    }

    private void showInfo(String message) { showDialog("Information", message); }
    private void showError(String message) { showDialog("Error", message); }

    private void showDialog(String title, String message) {
        Dialog d = new Dialog(this, title, true);
        d.setLayout(new BorderLayout());
        d.add(new Label("  " + message + "  "), Label.CENTER), BorderLayout.CENTER);
        Button ok = new Button("OK");
        ok.addActionListener(e -> d.dispose());
        Panel p = new Panel(new FlowLayout(FlowLayout.CENTER));
        p.add(ok);
        d.add(p, BorderLayout.SOUTH);
        d.setSize(420, 140);
        d.setLocationRelativeTo(this);
        d.setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if ("REG_SUBMIT".equals(e.getActionCommand())) {
            handleRegisterSubmit();
        }
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(EmployeeManagementApp::new);
    }
}
```